Build a complete PostgreSQL schema migration package for the housing allocations table, pgTAP isolation tests, and default-deny RLS policies in a downloadable script

Computer
Build an interactive compliance test runner dashboard for LTI 1.3 audit checks and PostgreSQL RLS security verification
Computer
Show me the full DDL for the housing allocations table
Walk me through the pgTAP assertion suite for isolation tests
Show how the default-deny RLS policies are structured

I created the downloadable migration package and a local interactive test-runner dashboard. It includes the housing DDL, default-deny/read-scoped RLS policies, 18 pgTAP assertions, 16 synthetic LTI checks, installation instructions, and a rollback script.

The archive's integrity is verified. PostgreSQL and Node execution have not been tested here; this is a reviewable implementation package, not a deployed system or compliance certification.

1. Package contents

The ZIP contains 12 files. The main migration and isolation-test scripts are also attached separately.

Component	Contents
Demo dependencies	Minimal institutional/person/cycle/bed tables for a disposable database
Housing migration	Full allocation table, foreign keys, overlap constraints, indexes, version trigger, RLS
Permission migration	Private identity bindings, constrained policy functions, tenant boundary, scoped read policy
pgTAP tests	Populated two-university fixtures and 18 assertions
Rollback	Explicit destructive rollback; never invoked automatically
LTI validator	Identified resource-link validation using `jose`
LTI test runner	16 generated-key, synthetic-token cases
Dashboard	Local UI for running tests, filtering results, and exporting JSON evidence
Documentation	Setup, dependencies, security boundaries, and remaining production requirements

Component	Contents
Checksums	SHA-256 manifest

Two important boundaries:

- The SQL identity adapter uses PostgreSQL `session_user`. It is not a Supabase `auth.uid()` implementation or a shared pooled-connection identity adapter.
- The dashboard runs actual local test scripts, but its LTI cases are synthetic—not a live LMS certification test.

2. Full housing DDL

The prerequisite tables must have the composite keys referenced below. The package includes optional demo prerequisites for an isolated sandbox.

```
BEGIN;

CREATE EXTENSION IF NOT EXISTS btree_gist;

CREATE TABLE semester.housing_allocations (
    tenant_id uuid NOT NULL,
    id uuid NOT NULL DEFAULT gen_random_uuid(),

    cycle_id uuid NOT NULL,
    bed_id uuid NOT NULL,
    person_id uuid NOT NULL,

    occupancy_range tstzrange NOT NULL,

    state text NOT NULL CHECK (
        state IN (
            'reserved',
            'offered',
            'contracted',
            'occupied',
            'completed',
            'cancelled',
            'expired'
        )
    ),

    expires_at timestamptz,

    request_key text NOT NULL CHECK (
        length(request_key) BETWEEN 1 AND 200
    ),

    created_at timestamptz NOT NULL DEFAULT now(),
    updated_at timestamptz NOT NULL DEFAULT now(),

    version bigint NOT NULL DEFAULT 1 CHECK (version >= 1),

    PRIMARY KEY (tenant_id, id),
```

```
    UNIQUE (tenant_id, request_key),

    FOREIGN KEY (tenant_id)
        REFERENCES semester.institutions (id),

    FOREIGN KEY (tenant_id, cycle_id)
        REFERENCES semester.housing_cycles (tenant_id, id),

    FOREIGN KEY (tenant_id, bed_id)
        REFERENCES semester.housing_beds (tenant_id, id),

    FOREIGN KEY (tenant_id, person_id)
        REFERENCES semester.people (tenant_id, id),

    CHECK (
        NOT isempty(occupancy_range)
        AND NOT lower_inf(occupancy_range)
        AND NOT upper_inf(occupancy_range)
        AND lower_inc(occupancy_range)
        AND NOT upper_inc(occupancy_range)
    ),

    CHECK (
        state <> 'reserved'
        OR expires_at IS NOT NULL
    ),

    CONSTRAINT housing_bed_overlap
    EXCLUDE USING gist (
        tenant_id WITH =,
        bed_id WITH =,
        occupancy_range WITH &&
    )
    WHERE (
        state IN (
            'reserved',
            'offered',
            'contracted',
            'occupied'
        )
    ),

    CONSTRAINT housing_person_overlap
    EXCLUDE USING gist (
        tenant_id WITH =,
        person_id WITH =,
        occupancy_range WITH &&
    )
    WHERE (
        state IN (
            'reserved',
            'offered',
            'contracted',
            'occupied'
        )
```

```
    )
);

CREATE INDEX housing_allocations_person_idx
ON semester.housing_allocations (
    tenant_id,
    person_id,
    state
);

CREATE INDEX housing_allocations_cycle_idx
ON semester.housing_allocations (
    tenant_id,
    cycle_id,
    state
);

CREATE INDEX housing_allocations_expiry_idx
ON semester.housing_allocations (
    tenant_id,
    expires_at
)
WHERE state = 'reserved';


CREATE FUNCTION semester.housing_allocation_version()
RETURNS trigger
LANGUAGE plpgsql
SET search_path = pg_catalog
AS $$
BEGIN
    IF NEW.tenant_id IS DISTINCT FROM OLD.tenant_id THEN
        RAISE EXCEPTION 'Allocation tenant cannot change'
            USING ERRCODE = '23514';
    END IF;

    NEW.created_at := OLD.created_at;
    NEW.updated_at := clock_timestamp();
    NEW.version := OLD.version + 1;

    RETURN NEW;
END;
$$;

REVOKE ALL ON FUNCTION
    semester.housing_allocation_version()
FROM PUBLIC;

CREATE TRIGGER housing_allocation_version
BEFORE UPDATE ON semester.housing_allocations
FOR EACH ROW
EXECUTE FUNCTION semester.housing_allocation_version();


ALTER TABLE semester.housing_allocations
    ENABLE ROW LEVEL SECURITY;
```

```
ALTER TABLE semester.housing_allocations
    FORCE ROW LEVEL SECURITY;

REVOKE ALL ON semester.housing_allocations FROM PUBLIC;

COMMIT;
```

The occupancy interval is `[start, end)`: one assignment can end exactly when another begins. Reservations and assignments share the same allocation boundary.

The DDL does not implement eligibility, selection windows, state-transition authority, or expiration workers. Those remain server-command responsibilities.

3. Default-deny RLS policies

Stage one: deny by default

The housing migration enables and forces RLS without adding policies. Ordinary users cannot see or modify rows.

PostgreSQL uses default deny when RLS is enabled and no applicable policy exists. Table privileges remain a separate requirement. [1]

Stage two: controlled reads

The policy migration introduces:

- `semester_ha_reader`: nonlogin, nonsuperuser, no `BYPASSRLS`.
- `semester_ha_policy`: narrowly privileged owner of identity-resolution functions.
- A private principal-binding table.
- A restrictive tenant boundary.
- A permissive read policy.

```
CREATE POLICY housing_tenant_boundary
ON semester.housing_allocations
AS RESTRICTIVE
FOR ALL
TO semester_ha_reader
USING (
    tenant_id = semester_ha_auth.current_tenant()
)
WITH CHECK (
    tenant_id = semester_ha_auth.current_tenant()
);

CREATE POLICY housing_read_scope
ON semester.housing_allocations
AS PERMISSIVE
FOR SELECT
```

```
TO semester_ha_reader
USING (
    semester_ha_auth.can_read(
        tenant_id,
        person_id
    )
);
```

Effective access

Principal binding	Result
No binding	No rows
Inactive binding	No rows
Expired binding	No rows
Active student	Own allocation rows in the bound institution
Active housing staff	Housing allocation rows in the bound institution
Another institution	No rows
Direct mutation	No grant

This reference gives housing staff institution-wide read scope. Building, cycle, team, or case restrictions require additional relationships.

Permissive policies combine with `OR`; restrictive policies combine with `AND`. The restrictive tenant policy therefore remains a boundary around the permitted reads. [1]

Identity safety

The helper functions look up `session_user` in an administrator-managed binding table. They do not trust a client-set tenant variable.

That works for the dedicated-session reference model. Before using Supabase or a pooled API, replace it with a verified session/JWT identity resolver and run equivalent behavioral tests.

No direct `INSERT`, `UPDATE`, `DELETE`, or `TRUNCATE` privileges are granted to the reader. PostgreSQL RLS does not govern `TRUNCATE`, and superusers or `BYPASSRLS` roles remain privileged. [1]

4. pgTAP assertion walkthrough

The suite uses populated university A/B fixtures rather than relying only on empty tables.

#	Assertion	Expected
1	RLS enabled and forced	Both flags true
2	Reader cannot bypass RLS	No superuser/BYPASSRLS
3	Expected policies only	Exactly two

#	Assertion	Expected
4	Reader cannot truncate	Privilege absent
5	Cross-tenant person reference	`23503`
6	Cross-tenant bed reference	`23503`
7	Overlapping bed occupancy	`23P01`
8	Overlapping person occupancy	`23P01`
9	Adjacent half-open occupancy	Succeeds
10	Unbound principal	Zero rows
11	Student's own allocation	One row
12	Student queries other institution	Zero rows
13	Student queries same-institution peer	Zero rows
14	Student directly updates allocations	`42501`
15	Student reads private identity bindings	`42501`
16	Housing staff reads assigned institution	Two rows
17	Housing staff queries other institution	Zero rows
18	Student binding revoked	Zero rows on next query

The test transaction rolls back fixtures and test roles.

Representative assertion

```
SELECT throws_ok(
    $sql$
    INSERT INTO semester.housing_allocations (
        tenant_id,
        cycle_id,
        bed_id,
        person_id,
        occupancy_range,
        state,
        request_key
    )
    VALUES (
        '<tenant-a>',
        '<tenant-a-cycle>',
        '<tenant-b-bed>',
        '<tenant-a-person>',
        '[2027-01-01,2027-02-01)',
        'offered',
        'cross-tenant-bed'
    )
    $sql$,
    '23503',
    NULL,
```

```
    'Cross-tenant bed FK rejected'
);
```

This illustration uses placeholders; the attached suite contains concrete UUID fixtures.

`throws_ok` checks the SQLSTATE, while `is` checks row counts and `lives_ok` checks that valid operations succeed. Using `NULL` for expected error text avoids dependence on localized PostgreSQL messages. [2]

Remaining tests

The suite does not prove:

- Concurrent reservation/confirmation behavior.
- Real nonprivileged login impersonation resistance.
- Supabase or pooled-API identity correctness.
- Staff building-level scope.
- Expired-reservation command handling.
- Storage, search, exports, or retention behavior.

The fixture harness uses a superuser-origin session to simulate identities. Test actual nonprivileged connections separately before production approval.

5. Interactive test dashboard

Implemented local functions

The dashboard provides:

- Run synthetic LTI checks.
- Run sandbox pgTAP tests.
- Pass/fail results.
- Failed-only filtering.
- JSON evidence export.
- Explicit test-coverage limitations.
- “Not run” and runner-error states.

The server uses fixed script names, no shell execution, localhost binding, origin/host checks, a random runner token, bounded output, and one active run at a time.

Database tests require explicit opt-in and a configured database name ending in `_semester_test`. That naming check is a guardrail—not proof that the database is disposable.

LTI test coverage

The 16 cases cover valid launch, issuer, audience, nonce, deployment, target, resource, ASCII subject, malformed/empty roles, expiration, signature, replay simulation, authorized party, mentor scope, and unknown contextual claims.

One correction to an earlier response: LTI permits an empty roles array. If the array is nonempty, it must contain at least one role from the defined vocabularies. The package allows the empty-array case.[3]

The checker implements a documented role-vocabulary subset. Extend the full vocabulary and perform real LMS, browser-state, key-rotation, and atomic replay tests before claiming interoperability.

Running the package

The README includes the exact installation sequence. For the supplied disposable demo:

```
psql "$TEST_DATABASE_URL" -v ON_ERROR_STOP=1 \
  -f sql/000_demo_dependencies.sql

psql "$TEST_DATABASE_URL" -v ON_ERROR_STOP=1 \
  -f sql/001_housing_allocations.sql

psql "$TEST_DATABASE_URL" -v ON_ERROR_STOP=1 \
  -f sql/002_read_policies.sql

pg_prove --verbose --dbname "$TEST_DATABASE_URL" \
  sql/003_isolation_tests.sql

npm install
npm run test:lti

ALLOW_SANDBOX_TESTS=yes TEST_PGSSLMODE=require npm start
```

Required dependencies include PostgreSQL, `btree_gist`, pgTAP, `pg_prove`, Node.js, and the npm dependency. Review and lock dependencies before deployment.

The dashboard is a functional local test runner, not yet a production governance monitor. Production integration still needs authenticated dashboard roles, isolated workers, persistent evidence, live audit/sync sources, institutional retention controls, and independent security review.

⁂

1. https://www.postgresql.org/docs/current/ddl-rowsecurity.html
2. https://pgtap.org/documentation.html
3. https://www.imsglobal.org/spec/lti/v1p3